31

FAISS & Vector Search

Searching millions of vectors in milliseconds

The fast engine behind semantic search and RAG

AI/ML Engineer Learning Series · Deep Dive Edition

What it is	A library for super-fast vector search
Full name	Facebook AI Similarity Search
The job	Find the closest vectors to a query, fast
Why needed	Comparing millions one-by-one is too slow
Your project	The retrieval index in your RAG chatbot

What's Inside

Here is everything covered in this guide, in order:

#	Section	What you'll learn
1	**The Search Problem**	Why finding nearest vectors is hard
2	**What is FAISS?**	The fast-search library
3	**Nearest Neighbor Search**	Finding the closest vectors
4	**Why Brute Force Fails**	The speed problem at scale
5	**How FAISS Stays Fast**	Indexing and approximate search
6	**Exact vs Approximate**	The speed-accuracy trade-off
7	**FAISS in Code**	Build an index and search it
8	**The RAG Connection**	How your chatbot uses it
9	**Common Mistakes**	What to avoid
★	**Cheatsheet**	Quick reference

1. The Search Problem

In the sentence transformers topic, you learned to turn text into vectors, where similar meanings sit close together. Semantic search means: given a question's vector, find the document vectors closest to it. Easy with a few documents. But what if you have millions?

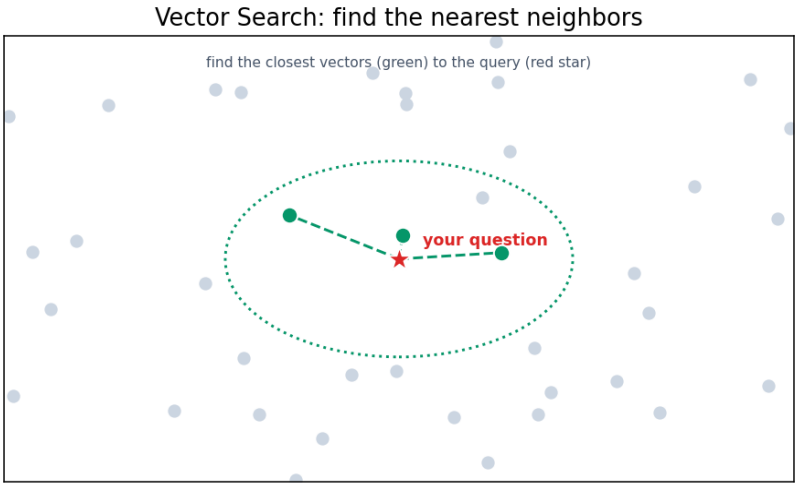

Fig 1 — Vector search: find the document vectors (green) closest to your question vector (red star).

The challenge is speed. To find the closest vectors, the obvious way is to measure the distance from your question to every single document, then pick the smallest. With a million documents, each having hundreds of numbers, that's a huge amount of math for every search. Too slow for a real app. FAISS solves this.

2. What is FAISS?

FAISS stands for Facebook AI Similarity Search. It's a free library, built by Meta, designed to do one thing extremely well: find the closest vectors to a query, blazingly fast, even among millions or billions of them. It's the standard tool for vector search.

Think of it as a super-smart filing system for vectors. Instead of checking every single document, FAISS organises the vectors cleverly so it can jump almost straight to the closest ones. This is what makes semantic search and RAG fast enough to feel instant.

> ■ *Your PDF RAG Chatbot uses FAISS as its retrieval index. When you ask it a question, FAISS is what instantly finds the relevant chunks from your PDF. This topic explains exactly how that works.*

3. Nearest Neighbor Search

The technical name for this task is nearest neighbor search: given a point (your query vector), find the nearest points (the most similar document vectors). 'Nearest' is measured by distance — usually the same cosine similarity idea from before, or plain straight-line distance.

You usually want the top-k nearest, like the 5 closest matches. FAISS returns these ranked by closeness, so the most relevant document comes first. In RAG, those top-k chunks become the context you feed to the LLM.

Term	Meaning
Query vector	The thing you're searching with (your question)
Nearest neighbor	The closest vector to the query
Top-k	The k closest matches (e.g. top 5)
Distance	How far apart two vectors are (closer = more similar)

4. Why Brute Force Fails

The simple approach — measure distance to every vector and sort — is called brute force. It's perfectly accurate, but it gets painfully slow as your data grows. Double the documents, double the work. With millions, every search takes too long.

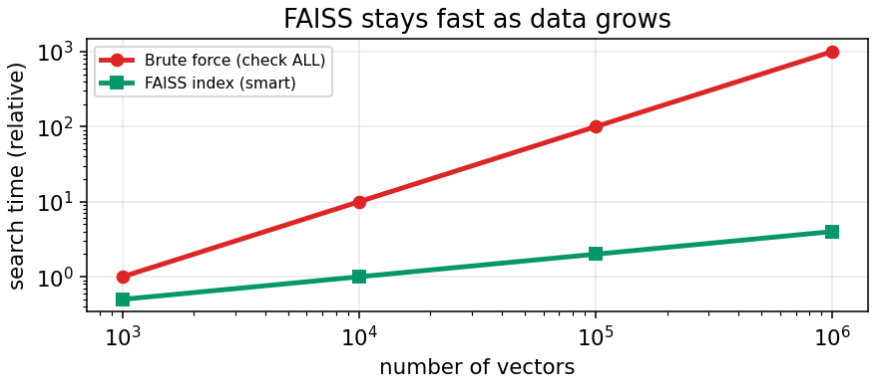

Fig 2 — Brute force slows down badly as data grows. FAISS uses smart indexing to stay fast.

For a handful of documents, brute force is fine — even a simple loop works. But a real app with a big knowledge base needs to search in milliseconds, every time, for many users. That's exactly the gap FAISS fills: it stays fast even as the data grows huge.

5. How FAISS Stays Fast

FAISS's secret is the index — a smart structure it builds over your vectors ahead of time. Instead of checking every vector, the index lets FAISS narrow down to a small promising region and only check those. It's like an index in a book: you don't read every page to find a topic, you jump to the right section.

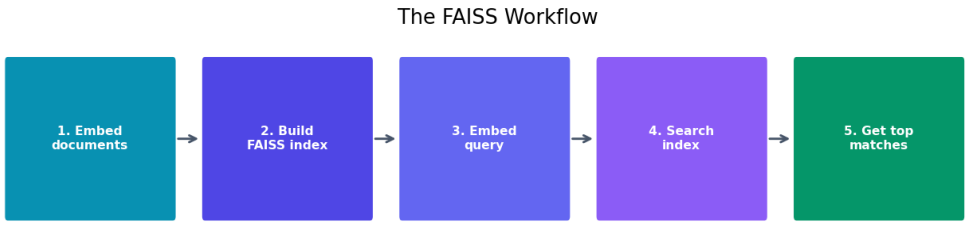

Fig 3 — The FAISS workflow: embed documents, build the index once, then embed and search queries fast.

The workflow is: build the index once from all your document vectors (a one-time setup), then search it as many times as you like, each search being lightning fast. The clever organisation done up front is what makes every later search quick.

6. Exact vs Approximate Search

FAISS offers a key choice. Exact search always finds the true nearest neighbors but is slower. Approximate search is willing to occasionally miss the very closest match in exchange for being much, much faster. For most uses, approximate is the smart pick.

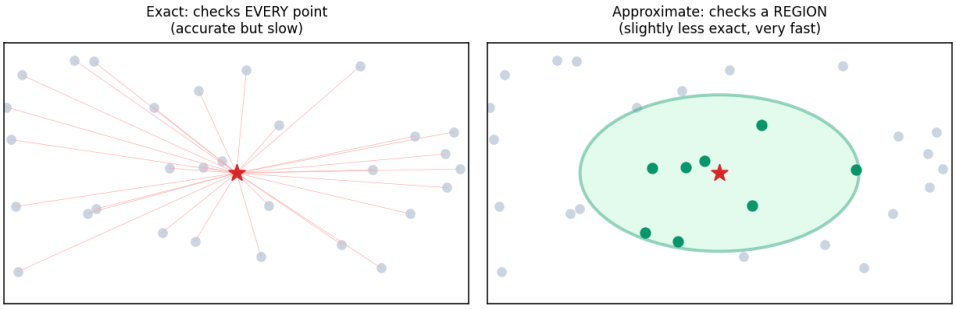

Fig 4 — Exact search checks every point. Approximate search checks a smart region — slightly less perfect, far faster.

	Exact search	Approximate search
Accuracy	Always perfect	Very good, rarely misses
Speed	Slower	Much faster
Best for	Small data, must be exact	Large data, speed matters
FAISS index type	IndexFlatL2	IndexIVFFlat, HNSW

■ *For most RAG apps, approximate search is perfect: missing the 5th-best chunk occasionally doesn't matter, and the speed gain is huge. Start with exact (IndexFlatL2) for small data, switch to approximate when you scale up.*

7. FAISS in Code

Here's the full flow: embed your text, build a FAISS index, and search it. This is essentially your RAG chatbot's retrieval core.

```
# pip install faiss-cpu sentence-transformers
import faiss
import numpy as np
from sentence_transformers import SentenceTransformer

model = SentenceTransformer('all-MiniLM-L6-v2')

# 1. Your documents -> vectors
docs = ['cats are pets', 'dogs are loyal', 'python is code', ...]
vectors = model.encode(docs).astype('float32')

# 2. Build a FAISS index
dim = vectors.shape[1]          # e.g. 384
index = faiss.IndexFlatL2(dim)  # exact search index
index.add(vectors)              # add all document vectors

# 3. Search with a query
query = model.encode(['tell me about cats']).astype('float32')
distances, ids = index.search(query, k=3)   # top 3

# 4. Show results
for i in ids[0]:
    print(docs[i])

# Save / load the index
faiss.write_index(index, 'my.index')
index = faiss.read_index('my.index')
```

■ *Two details: FAISS wants float32 vectors (use .astype('float32')), and index.search returns both distances and the IDs (positions) of the matches. Use those IDs to look up the original text.*

8. The RAG Connection

FAISS is the 'search' step in your RAG pipeline. Here's where it fits in the whole flow you've been building across topics:

- **Setup (once):** split the PDF into chunks, embed each chunk (sentence transformer), add all vectors to a FAISS index.
- **Per question:** embed the question, then FAISS finds the top-k closest chunks in milliseconds.
- **Answer:** those chunks become the context, and the LLM (via Groq) writes the final answer from them.

So FAISS is the high-speed matchmaker between the question and the right pieces of your document. Without it, searching a large PDF or knowledge base would be too slow to be useful. It turns semantic search from a neat idea into a practical, instant tool.

■ *Your chatbot used hybrid retrieval: FAISS (meaning-based) combined with BM25 (keyword-based). FAISS handles the semantic matching; BM25 catches exact keyword matches FAISS might rank lower. Together they retrieve better than either alone.*

9. Common Mistakes

Mistake	Why it's a problem	Fix
Wrong vector data type	FAISS errors out	Use .astype('float32')
Different embed model for query	Vectors don't match	Same model for docs and query
Rebuilding index every search	Very slow	Build once, save, reuse
Mismatched dimensions	Index add/search fails	Index dim must match vectors
Exact search on huge data	Too slow	Use an approximate index
Losing the text mapping	IDs are useless alone	Keep docs list to map IDs back

Quick Reference Cheatsheet

Concept / Task	Meaning or Code
Vector search	find the closest vectors to a query
Nearest neighbor	the most similar vector
Top-k	the k closest matches
Brute force	check every vector (slow at scale)
Index	smart structure for fast search
Exact index	faiss.IndexFlatL2(dim)
Approximate index	IndexIVFFlat, HNSW (faster)
Install	pip install faiss-cpu
Build index	index = faiss.IndexFlatL2(dim)
Add vectors	index.add(vectors) # float32
Search	index.search(query, k=5)
Save / load	faiss.write_index / read_index
Pairs with	sentence transformers, RAG

The one idea to remember: FAISS finds the closest vectors to a query incredibly fast by building a smart index instead of checking every vector. It's what makes semantic search and RAG quick enough to be practical, even with millions of documents.

Next Topic: Vector Databases — the next step up from FAISS: full systems that store, manage, update, and search your vectors, with all the production features a real app needs.